Instituto Superior de Engenharia do Porto

Big Data Tools I

Relatório Técnico: Rede Social "Libri"

Março de 2026

Cátia Costa - 1250457

Manuel Sousa - 1250465

Patrícia Faria - 1250471

Índice

1. Introdução 3
2. Repositório 3
3. Arquitetura de Armazenamento e Justificação 3
4. Modelação de Dados e Padrões de Acesso 5
A. MongoDB 5
B. Cassandra 6
C. Redis 8
5. Operações e queries principais 9
6. Índices, Chaves e TTL 9
A. MongoDB 9
B. Redis 10
7. Interpretação de Resultados e Métricas 10
A. MongoDB 10
B. Cassandra 12
C. Redis 14
8. Integração e Fluxo de Dados entre Sistemas NoSQL 17

1. Introdução

O Libri é o projeto de uma mini rede social para leitores. A plataforma permite que os utilizadores criem perfis, partilhem as suas leituras atuais, atribuam pontuações (ratings) a livros e interajam com o conteúdo de outros utilizadores através de um sistema de seguidores e reações em tempo real (likes e comentários).

Neste projeto foi apenas desenhado o modelo de dados, não tendo sido desenvolvida a aplicação (backend/ frontend). No entanto, o modelo foi pensado para que futuramente se possa avançar para o desenvolvimento aplicacional.

Para suportar as funcionalidades core da rede social, a arquitetura foi desenhada em torno de três bases de dados: MongoDB, Cassandra e Redis.

Através da integração destas tecnologias, o projeto Libri demonstra como a orquestração de diferentes motores NoSQL permite criar uma plataforma robusta, capaz de escalar horizontalmente e manter tempos de resposta constantes perante o crescimento do volume de dados.

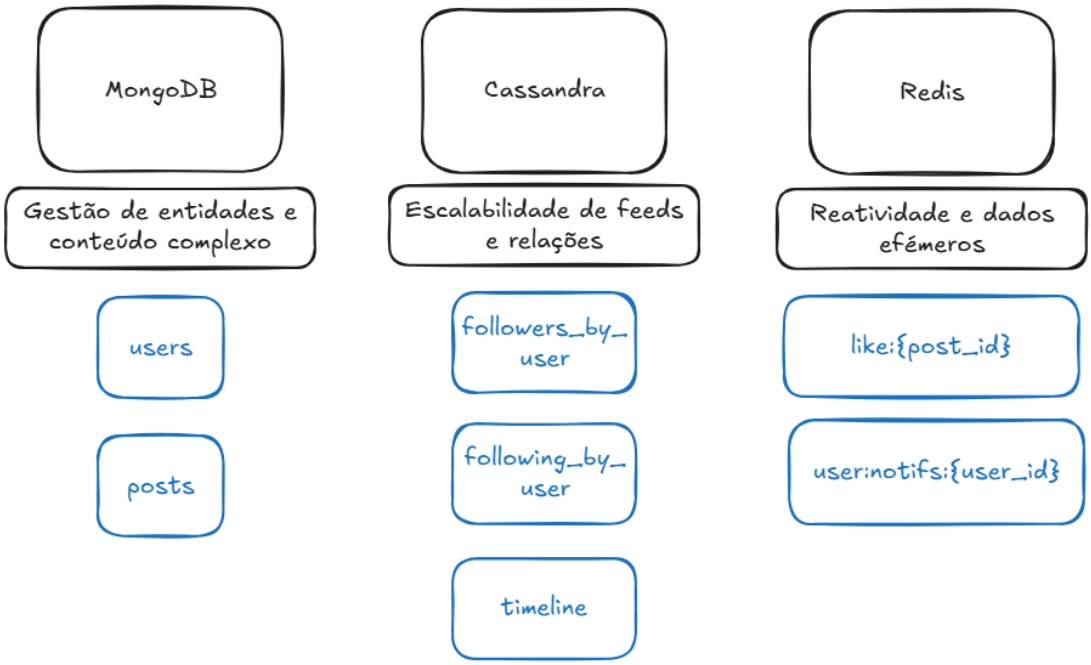
2. Repositório

De forma a garantir a reprodutibilidade do projeto, foi disponibilizado um repositório GitHub com os scripts de inicialização, dados de teste e instruções de execução dos fluxos implementados: https://github.com/CatiaCosta95/BDTI_Libri/tree/main

3. Arquitetura de Armazenamento e Justificação

A arquitetura de dados da rede social "Libri" foi desenhada para utilizar cada tecnologia de base de dados para a função em que pode apresentar maior eficiência. Esta abordagem permite maximizar a escalabilidade horizontal e minimizar a latência percebida pelo utilizador final.

A solução utiliza três bases de dados distintas:



- **MongoDB (Document Store):** funciona como o sistema de registo principal para as entidades de negócio (**Utilizadores** (**Perfis)** e **Publicações**.
    - O modelo documental (JSON/BSON) permite armazenar estruturas ricas e embebidas (nested), como o perfil do utilizador (biografia, metas de leitura) e os comentários de um post, num único documento.
    - Facilita consultas complexas e agregações, como pesquisas de texto em títulos de livros ou cálculos de estatísticas de leitura, que seriam pesadas noutras bases de dados NoSQL.
    - A natureza schemaless do MongoDB permite a evolução rápida do modelo de dados sem necessidade de migrações rígidas de schema (flexibilidade).

- **Cassandra (Wide Column Store):** utilizado para gerir as relações de seguidores e timeline (feed temporal).
    - Sendo uma base de dados Wide Column Store, o Cassandra é ideal em cenários de escrita massiva e descentralizada. A sua arquitetura garante que não existam pontos únicos de falha (disponibilidade).
    - Estratégia de *Fan-out*: utiliza-se a desnormalização para criar tabelas específicas para cada query (Query-First Design). Ao inserir um post, este é replicado para as partições de timeline de todos os seguidores, garantindo que a leitura do feed seja

uma operação de procura simples em disco, sem necessidade de cruzamentos de tabelas em tempo real.

- **Redis (Key-Value Store):** atua como a camada de gestão de dados em tempo real para likes e notificações.
    - Como os dados são armazenados na memória RAM, o Redis oferece latências sub-milisegundo. É ideal para contadores de alta frequência (likes) que seriam ineficientes se escritos constantemente em disco.
    - Através de estruturas como *Sorted Sets*, as notificações são ordenadas automaticamente por timestamp. O uso de TTL (Time To Live) permite que as notificações antigas expirem automaticamente, mantendo a base de dados "limpa" sem intervenção manual.

Tabela Resumo:

Tecnologia	Modelo de Dados	Função no Sistema	Padrão de Acesso Principal	Justificação Técnica
MongoDB	Document Store (BSON)	*System of Record*	Leitura/Escrita de documentos ricos	Flexibilidade de esquema para perfis e posts com comentários embebidos (*Embedding*).
Apache Cassandra	Wide column store	Gestão de *Feeds*	Escrita massiva e leitura sequencial	Alta disponibilidade e performance no *Fan-out* de publicações para as *timelines* de seguidores.
Redis	Chave-Valor (*In-memory*)	Interações em Tempo Real	Incrementos atómicos (Likes)	Latência sub-milisegundo para dados de alta frequência e voláteis.

4. Modelação de Dados e Padrões de Acesso

A. MongoDB

Collection: users

- **Padrão Documento Único (Perfis):** Optou-se por embutir (embedding) o objeto profile dentro da collection users. Isto permite que, ao carregar os dados de conta, a bio e as

preferências de leitura sejam recuperadas numa única operação de I/O, eliminando a necessidade de cruzamentos de dados complexos para o perfil/ procuras extra (lookups).

Collection: posts

- **Padrão Embedding (Comentários do post):** Os comentários foram modelados como um array de subdocumentos dentro de cada post.
    - Vantagem: Numa rede social, o padrão de acesso comum é "Ler Post → Ler Comentários". Ao estarem embebidos, o post e toda a sua conversa são recuperados numa única leitura física.
    - Denormalização: Incluímos o username dentro de cada comentário. Isto evita que o servidor tenha de fazer um "lookup" à collection users apenas para mostrar quem escreveu o texto.
    - Como se trata de uma mini rede social, o limite de 16MB por documento do MongoDB é mais do que suficiente para centenas de comentários por post.

B. Cassandra

O Cassandra não atua como sistema de registo principal, mas sim como uma camada derivada e otimizada para leitura, contendo uma versão desnormalizada dos dados originalmente armazenados no MongoDB.

O Cassandra foi utilizado para suportar padrões de acesso de baixa latência numa mini rede social, em particular o feed temporal e as relações de seguidores. A sua escolha prende-se com a necessidade de garantir elevada disponibilidade, escalabilidade horizontal e desempenho consistente em cenários com grande volume de escrita e leitura.

A modelação foi realizada com base numa abordagem orientada às queries (*Query-First Design*), onde cada tabela foi desenhada para responder diretamente a um padrão de acesso específico. Esta abordagem é fundamental no Cassandra, uma vez que não existe suporte para joins ou agregações complexas em tempo real.

A inserção de dados na tabela timeline segue uma lógica de *fan-out on write*, onde cada nova publicação é replicada para as timelines dos seguidores do autor. Este processo permite que o feed já se encontre pré-calculado no momento da leitura, evitando operações dispendiosas em tempo real.

Neste contexto, a tabela timeline suporta a leitura eficiente do feed de um utilizador, enquanto as tabelas followers_by_user e following_by_user suportam consultas sobre quem segue um utilizador e quem esse utilizador segue. Esta escolha está alinhada com o requisito de modelação orientada às queries, com definição explícita de *partition key* e *clustering key*.

A tabela timeline foi criada para suportar o feed de cada utilizador, permitindo recuperar de forma eficiente as publicações mais recentes das pessoas que segue.

A chave primária é composta por:

- **Partition Key:** user_id
- **Clustering Key:** post_id (do tipo TimeUUID)

O user_id permite agrupar todas as publicações relevantes de um utilizador na mesma partição, enquanto o post_id, sendo um *timeuuid*, garante ordenação temporal automática. Foi definida uma ordenação descendente (WITH CLUSTERING ORDER BY (post_id DESC)), permitindo que os posts mais recentes sejam devolvidos primeiro.

Esta estrutura permite que a leitura do feed seja realizada de forma sequencial e altamente eficiente, sem necessidade de ordenação adicional ou processamento complexo. A utilização de LIMIT nas queries possibilita ainda a implementação de paginação, simulando o comportamento real de uma aplicação, onde apenas um subconjunto das publicações é carregado de cada vez.

Para suportar as relações entre utilizadores, foram criadas duas tabelas distintas: followers_by_user e following_by_user. A primeira responde ao padrão de acesso "Quem são os seguidores do utilizador X?", enquanto a segunda suporta a query "Quem é que o utilizador X segue?". Em ambos os casos, o user_id foi definido como *partition key*, permitindo agrupar as relações de cada utilizador na mesma partição. Já o follower_id e o followed_id funcionam como *clustering keys*, organizando e identificando unicamente cada relação dentro dessa partição.

Adicionalmente, foi adotada uma estratégia de denormalização, incluindo campos como follower_username e followed_username, de modo a evitar consultas adicionais ao MongoDB e permitir leituras mais eficientes.

Tabela: followers_by_user

- **Padrão de Acesso:** "Quem são os seguidores do Utilizador X?"
- **Chave Primária:** PRIMARY KEY (user_id, follower_id)
    - **Partition Key:** user_id, permitindo agrupar todos os seguidores de um utilizador na mesma partição.
    - **Clustering Key:** follower_id, permitindo organizar e identificar unicamente cada seguidor dentro da partição.
- **Dados Denormalizados:** foi incluído o campo follower_username, de forma a permitir apresentar a lista de seguidores sem necessidade de consultar o MongoDB.

Tabela: following_by_user

- **Padrão de Acesso:** "Quem é que o Utilizador X segue?"
- **Chave Primária:** PRIMARY KEY (user_id, followed_id)
    - **Partition Key:** user_id, permitindo agrupar todas as relações de seguimento do utilizador na mesma partição.
    - **Clustering Key:** followed_id, permitindo organizar e identificar unicamente cada utilizador seguido dentro da partição.

C. Redis

O Redis é utilizado como uma camada de suporte para operações que exigem rapidez, fluidez e que não precisam de armazenamento a longo prazo.

Likes (Hashes):

- **Sets (SADD)**: Criámos a chave like:pX:users para registar o username de cada utilizador que interagiu com o post pX. Desta forma, garantimos a unicidade da operação: o Redis ignora automaticamente tentativas de "likes" duplicados do mesmo utilizador.
- **Hashes (HINCRBY)**: Paralelamente, mantemos um contador atómico na chave like:pX. Assim, guardamos apenas o número final, logo temos um acesso mais instantâneo e que gasta menos memória do que processar um set.
- **Justificação:** Evita o *overhead* de escrita em disco do MongoDB para uma operação de alta frequência e baixa complexidade, como por exemplo, os likes. O comando HINCRBY é atómico e executado em memória, suportando milhares de interações por segundo com pouca latência.

Notificações (Sorted Sets):

- Armazenamos as últimas notificações de cada utilizador num Sorted Set. O score do set é o timestamp da interação, garantindo que a aplicação lê sempre as notificações da mais recente para a mais antiga através do comando ZREVRANGE.
- **Comando:** ZADD user:notifs:{username} {timestamp} {payload}
- **TTL (Time To Live):** O comando EXPIRE "user:notifs:user1" 604800 define um TTL (Time To Live) de 7 dias (604,800 segundos) para as notificações desse utilizador. Assim, as notificações antigas expiram sozinhas para libertar memória RAM, pois no Redis a memória RAM é mais cara e limitada.

Ranking de livros:

- Também utilizámos Sorted Sets para gerar um ranking dinâmico de top dos livros.
- **Comando:** ZINCRBY. O sistema incrementa o score do livro correspondente no Redis.
- **Comando:** ZREVRANGE. Permite recuperar instantaneamente o Top dos livros mais populares com uma baixa complexidade, garantindo que a funcionalidade de "Tendências" do Libri seja fluida.

5. Operações e queries principais

Os scripts de criação, inserts e queries estão nos documentos listados abaixo:

mongobd_libri.js

console_cassandra_social_libri_1.3.sql

redis_libri 2.sql

6. Índices, Chaves e TTL

A. MongoDB

Para garantir que o sistema escala, foram implementados os seguintes índices:

- **Unique Index (username):** Garante a integridade dos dados ao nível da base de dados, impedindo duplicados.

  db.users.createIndex({ username: 1 }, { unique: true });

- **Compound Index (author_id: 1, created_at: -1):** Este índice é vital para o padrão de acesso "Ver Perfil do Utilizador". Ele permite listar os posts de um autor específico já ordenados do mais recente para o mais antigo de forma instantânea.

  db.posts.createIndex({ author_id: 1, created_at: -1 });

- **Text Index (book_title, author_book):** Implementado para permitir pesquisas flexíveis. Ao contrário de um índice normal, o índice de texto permite encontrar "Duna" mesmo que o utilizador pesquise por termos parciais ou associados, suportando a funcionalidade de descoberta da rede.

  db.posts.createIndex({ book_title: "text", author_book: "text" });

B. Redis

- **TTL (Time-To-Live):** um mecanismo que implementamos para garantir que o sistema não cresce indefinidamente com dados pouco relevantes, otimizando o uso de memória RAM e armazenamento em disco. No Redis, implementámos a expiração automática nas notificações ao fim de 7 dias. Assim, o Redis elimina-as automaticamente após este período, libertando espaço sem ser preciso intervenção manual.

7. Interpretação de Resultados e Métricas

A. MongoDB

Com base nas queries de agregação executadas, os resultados demonstram a eficiência do modelo:

- Métrica top comentadores:

```
// Top Comentadores - Quem são os utilizadores que mais comentam?

db.posts.aggregate([
  { $unwind: "$comments" }, // Parte o array em documentos individuais
  { $group: {
      _id: "$comments.username",
      total_comentarios: { $sum: 1 }
  }},
  { $sort: { total_comentarios: -1 } },
  { $limit: 5 }
]);
```

Output | Esta query analisa o...array de comentários.

2 rows

	{} _id	{} total_comentarios
1	diogo_viana	1
2	carla_books	1

Esta query analisa os comentários para ver quem são os utilizadores mais ativos a comentar na rede. Ela usa o $unwind para "desconstruir" o array de comentários.

<u>Resultado:</u> Permite identificar em tempo real os utilizadores mais ativos para algoritmos importantes, como por exemplo destaque na rede.

- Métrica de objetivos de leitura:

```
//Métrica de Objetivos - Relaciona os livros concluídos com o objetivo de leitura do perfil do utilizador
db.users.aggregate([
  {
    $lookup: {
      from: "posts",
      localField: "_id",
      foreignField: "author_id",
      as: "meus_posts"
    }
  },
  {
    $project: {
```

Output | Relaciona os livros ... perfil do utilizador

3 rows

	{} _id	{} livros_concluidos	{} objetivo	{} username
1	69bd9e1b04de56640b990644	0	12	bruno_tech
2	69bd9e1b04de56640b990645	1	30	carla_books
3	69bd9e1b04de56640b990646	0	15	diogo_viana

O uso de $lookup aliado a $filter permite processar esta lógica de negócio complexa (comparar posts concluídos vs meta de leitura) diretamente no motor da base de dados.

Resultado: Isto reduz o volume de dados transferidos para o backend, enviando apenas o resultado final calculado.

- Métrica para análise de tendências locais:

```
//Pesquisa geográfica por tendência - Quais os livros mais populares por localização?
db.users.aggregate([
  {
    $lookup: {
      from: "posts",
      localField: "_id",
      foreignField: "author_id",
      as: "posts_info"
    }
  },
  { $unwind: "$posts_info" },
  {
    $group: {
```

Output | Pesquisa geográfica ...ares por localização?

2 rows

	{} _id	{} contagem
1	{"local": "Coimbra", "livro": "Cem Anos de Solidão"}	1
2	{"local": "Lisboa", "livro": "Duna"}	1

Esta pesquisa demonstra a capacidade de cruzar dados geográficos dos utilizadores com as suas publicações.

Resultado: Identifica padrões como "Qual o livro mais lido em Lisboa?". Numa aplicação real, esta métrica pode orientar na recomendação de conteúdos e publicidade direcionada.

B. Cassandra

No Cassandra, os resultados observados refletem a modelação orientada às queries adotada no sistema, permitindo garantir operações eficientes e com baixa latência no contexto de uma rede social.

Foram realizadas operações nas tabelas followers_by_user e following_by_user, permitindo responder diretamente às queries “Quem segue o utilizador X?” e “Quem é que o utilizador X segue?”, sem necessidade de cruzamento de dados.

- Consulta dos seguidores de um utilizador na tabela followers_by_user.

```
// Seguidores da Carla - "Quem segue a Carla?"
SELECT * FROM followers_by_user
WHERE user_id = 11111111-1111-1111-1111-111111111111;
```

Output | Seguidores da Carla ..."Quem segue a Carla?"

	user_id	follower_id	followed_at	follower_username
1	11111111-1111-1111-1111-1…	22222222-2222-2222-2222-2…	2026-03-24 15:31:21.457	bruno_tech

- Consulta dos utilizadores seguidos por um utilizador.

```
// Seguidos do Bruno - "Quem o Bruno segue?"
SELECT * FROM following_by_user
WHERE user_id = 22222222-2222-2222-2222-222222222222;
```

Output | Seguidos do Bruno - ..."Quem o Bruno segue?"

	user_id	followed_id	followed_at	followed_username
1	22222222-2222-2222-2222-222222222222	11111111-1111-1111-1111-111111111111	2026-03-24 15:31:31.119	carla_books

Foram também inseridas publicações na tabela timeline, simulando o funcionamento de um feed. Cada publicação é associada ao user_id do utilizador que irá visualizar o conteúdo, refletindo a estratégia de *fan-out on write*.

- Inserção de publicações na tabela timeline simulando o feed.

```sql
// Inserir post (1) da Carla no feed da Patrícia.
INSERT INTO timeline (
  user_id, post_id, author_id, author_username,
  book_title, author_book, content, rating, status, created_at
) VALUES (
  user_id 22222222-2222-2222-2222-222222222222,
  post_id now(),
  author_id 11111111-1111-1111-1111-111111111111,
  author_username 'carla_books',
  book_title 'Duna',
  author_book 'Frank Herbert',
  content 'A política e a ecologia de Arrakis são fascinantes.',
  rating 5,
  status 'Lendo',
  created_at toTimestamp(now())
);
SELECT *
FROM timeline
WHERE user_id = 22222222-2222-2222-2222-222222222222;
```

Output | timeline

	user_id	post_id	author_book	author_id
1	22222222-2222-2222-2222-222222222222	5ba43870-2797-11f1-a2ce-33779e4947f2	Frank Herbert	11111111-1111-1111-1

Ao executar queries sobre a tabela timeline, observa-se que os resultados são devolvidos já ordenados temporalmente, do mais recente para o mais antigo. Isto deve-se ao uso de timeuuid como clustering key e à ordenação descendente definida na tabela. Esta organização permite que a leitura seja realizada de forma sequencial e eficiente, sem necessidade de ordenação adicional.

- Consulta do feed de um utilizador ordenado temporalmente.

```sql
//Posts presentes no feed do Bruno:
SELECT post_id, author_username, book_title, status, created_at
FROM timeline
WHERE user_id = 22222222-2222-2222-2222-222222222222;
```

Output | Posts presentes no feed do Bruno:

	post_id	author_username	book_title	status	created_at
1	b52594c0-2797-11f1-a2ce-33779e4947f2	carla_books	Orgulho e Preconceito	Quero Ler	2026-03-24 15:39:5
2	b51f7a40-2797-11f1-a2ce-33779e4947f2	carla_books	1984	Concluído	2026-03-24 15:39:5
3	b446cd80-2797-11f1-a2ce-33779e4947f2	carla_books	Duna	Lendo	2026-03-24 15:39:5

Para simular um feed real, foi utilizada a operação `LIMIT` sobre a tabela `timeline`, permitindo obter apenas as publicações mais recentes de um utilizador. Esta abordagem é compatível com paginação e com leitura eficiente em Cassandra, dado que os dados já se encontram organizados por `user_id` e por `timeuuid` em ordem decrescente.

- Consulta do feed com limitação de resultados (paginação).

```
//Feed realista - Na app não queremos carregar os posts todos. Carrega no feed posts ordenados pelo post_id até um limite.
SELECT post_id, author_username, book_title, status, created_at
FROM timeline
WHERE user_id = 22222222-2222-2222-2222-222222222222
LIMIT 2;
```

Output | Feed realista - Na a...ost_id até um limite.

	post_id	author_username	book_title	status	created_at
1	b52594c0-2797-11f1-a2ce-33779e4947f2	carla_books	Orgulho e Preconceito	Quero Ler	2026-03-24 15:3
2	b51f7a40-2797-11f1-a2ce-33779e4947f2	carla_books	1984	Concluído	2026-03-24 15:3

Do ponto de vista de desempenho, o Cassandra apresenta tempos de resposta consistentes, uma vez que as queries são simples e diretamente suportadas pela modelação. A ausência de joins e agregações contribui para a previsibilidade das operações.

Relativamente à escrita, o sistema mantém um bom desempenho mesmo quando uma publicação é replicada por múltiplos utilizadores, no contexto do *fan-out on write*. Isto deve-se ao modelo de escrita do Cassandra, baseado em operações sequenciais (append), que permite lidar eficientemente com grandes volumes de dados.

A estratégia de denormalização adotada, com a inclusão de campos como usernames diretamente nas tabelas, permite evitar consultas adicionais ao MongoDB, tornando as operações mais rápidas e reduzindo a complexidade.

De forma geral, os resultados obtidos demonstram que o Cassandra é adequado para suportar o feed de uma rede social, garantindo baixa latência, boa escalabilidade e desempenho consistente, mesmo com o crescimento do número de utilizadores e publicações.

Este fluxo foi simulado manualmente através de inserts na tabela timeline, representando o comportamento esperado do sistema numa aplicação real.

C. Redis

Começamos por fazer um registo de quem deu um like em que post (SADD) e ao mesmo tempo ter um contador de likes para os posts (HINCRBY).

Além do mais, asseguramo-nos que a mesma pessoa não pode dar mais do que um like na mesma publicação.

```
-- Registo de "Quem deu like no quê" e contador de likes
SADD "like:p1:users" "carla_books"
HINCRBY "like:p1" "count" 1
SADD "like:p1:users" "diogo_viana"
HINCRBY "like:p1" "count" 1
SADD "like:p2:users" "bruno_tech"
HINCRBY "like:p2" "count" 1

-- A mesma pessoa não pode dar mais do que 1 like, logo aqui vai dar 0
SADD "like:p1:users" "carla_books"
```

Output | Result 56-6 | Result 57 | Result 56-2 | Result 56-3

	value
1	0

O comando SMEMBERS lista os utilizadores que interagiram com o post, enquanto o HGET mostra-nos o valor atual do número de likes do post de forma mais eficiente.

Do ponto de vista de latência, o HGET acede a um valor já calculado, o que permite à aplicação exibir o número de interações instantaneamente. Ao manter um contador atómico via Hash, reduzimos a carga computacional, garantindo que a interface do utilizador responde com latência sub-milisegundo, independentemente de o post ter 10 ou 1 milhão de likes."

```
-- Saber quantos likes tem o post
--retorna quem deu likes
SMEMBERS "like:p1:users"
```

Output | Result 56-6 | Result 57

	value
1	carla_books
2	diogo_viana

```
--retorna o nr de likes (mais rapido)
HGET "like:p1" "count"
```

Output | Result 56-6 | Result 57

	value
1	2

Podemos observar a estrutura de notificações armazenada em memória. Cada entrada é um objeto JSON que contém o contexto da interação.

Com o timestamp de cada notificação e através do comando ZREVRANGE garantimos que as notificações aparecem ao utilizador por ordem cronológica do mais recente para o mais antigo.

O comando EXPIRE mostra o tempo restante de vida da chave de notificações. Este mecanismo é vital para a sustentabilidade da arquitetura, garantindo que dados efémeros são eliminados automaticamente após 7 dias, prevenindo o esgotamento da memória RAM.

```
-- notifica o Bruno que a Carla e o Diogo deram like
ZADD "user:notifs:bruno_tech" 1710950000 "{\"type\": \"like\", \"from\": \"carla_books\", \"post\": \"Duna\"}"
ZADD "user:notifs:bruno_tech" 1710951000 "{\"type\": \"like\", \"from\": \"diogo_viana\", \"post\": \"Duna\"}"
EXPIRE "user:notifs:bruno_tech" 604800
-- notifica a Carla que o Bruno deu like
ZADD "user:notifs:carla_books" 1710970000 "{\"type\": \"like\", \"from\": \"bruno_tech\", \"post\": \"Cem Anos de
EXPIRE "user:notifs:carla_books" 604800

--vai listar as notificações que são apresentadas e apresenta a partir da mais recente
ZREVRANGE "user:notifs:bruno_tech" 0 -1 WITHSCORES
tifs:carla_books" 0 -1 WITHSCORES
```

Output | Result 67 | JSON

	value	score
1	{"type": "like", "from": "diogo_viana", "post": "Duna"}	1710951000
2	{"type": "like", "from": "carla_books", "post": "Duna"}	1710950000

Visualização do ranking dinâmico dos livros. O Redis processa o peso de cada livro em tempo real usando ZINCRBY. Este top permite extrair tendências de leitura da rede social instantaneamente, sem necessidade de percorrer todos os documentos de posts no MongoDB.

```
-- Ranking dos livros
ZINCRBY "ranking:livros" 7 "Duna"
ZINCRBY "ranking:livros" 5 "Cem Anos de Solidão"
ZINCRBY "ranking:livros" 10 "1984"

-- Top dos Livros, descrscente
ZREVRANGE "ranking:livros" 0 -1 WITHSCORES
```

Output | Result 71 | Result 72

	value	score
1	1984	10
2	Duna	7
3	Cem Anos de Solidão	5

8. Integração e Fluxo de Dados entre Sistemas NoSQL

Para melhor explicar o fluxo de dados entre sistemas, será abordado o fluxo de criação de uma publicação.

O processo de criação de uma publicação segue uma arquitetura orquestrada pelo Backend (não desenvolvido neste projeto), garantindo que cada base de dados desempenhe a função para a qual foi otimizada.

Quando um utilizador cria uma nova publicação, inicia-se o fluxo com a gravação no MongoDB. O Backend envia o conteúdo rico da publicação (título do livro, rating, texto) e o motor documental gera automaticamente um ObjectId único no campo _id.

O Backend recupera este ObjectId e utiliza-o como a referência única para as operações seguintes, garantindo que os dados no Cassandra e Redis apontem sempre para o documento original no MongoDB.

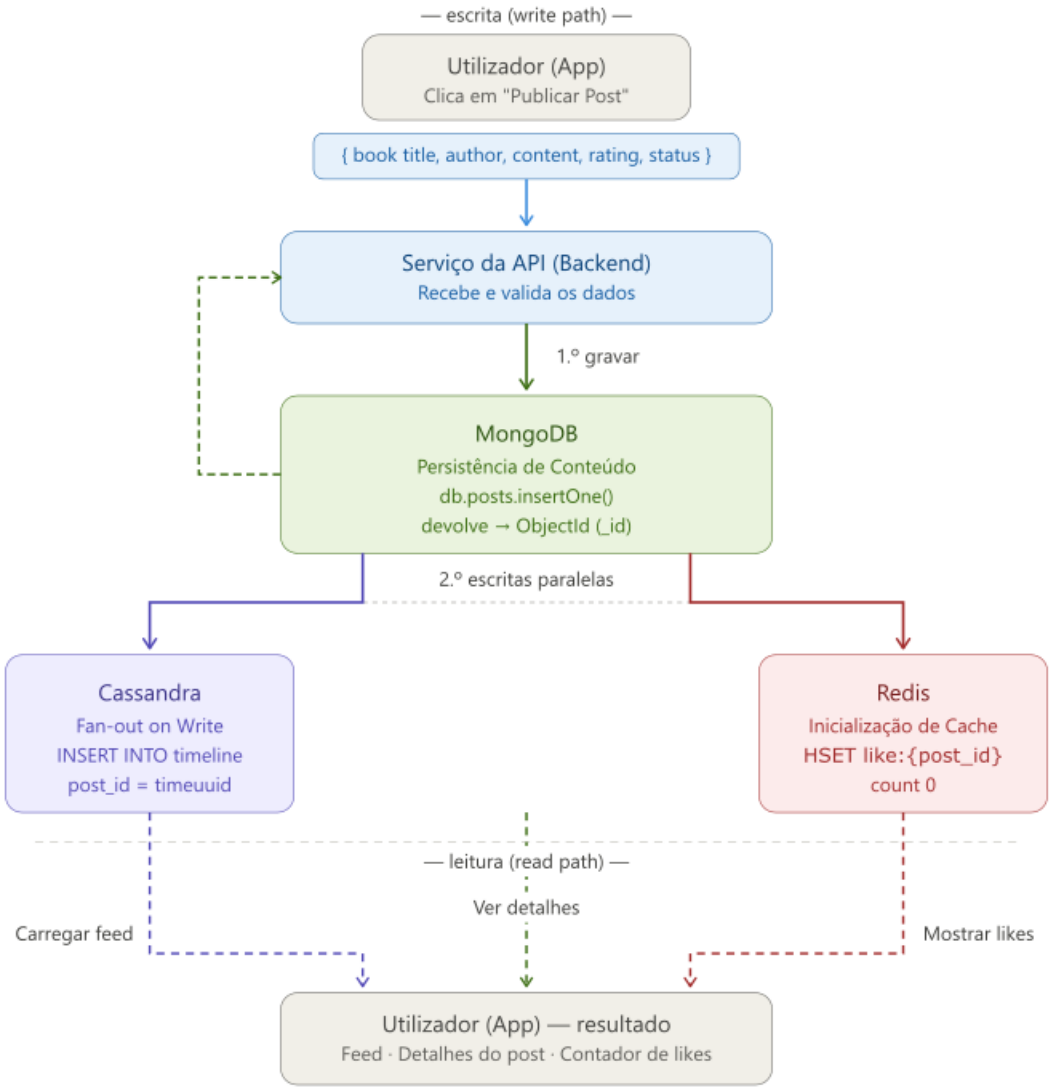
Este identificador é depois utilizado como referência nas restantes operações, garantindo consistência entre os diferentes sistemas. Após a confirmação da escrita no MongoDB, o Backend executa o processo de *fan-out on write*, convertendo o identificador para um formato compatível com o Cassandra (*timeuuid*) e replicando a publicação nas tabelas de *timeline* de todos os seguidores do utilizador.

Assim, em vez de fazer pesquisas complexas quando um utilizador abre a App Libri, o Cassandra já tem o feed pré-calculado e ordenado fisicamente no disco. A leitura é, por isso, ultrarrápida e sequencial.

Em paralelo à escrita no Cassandra, o Backend acede ao Redis para inicializar o estado de interação da publicação. É criado um Hash com a chave like:{post_id}, definindo o contador count a zero. Como as operações de alta frequência, como "Likes", não devem sobrecarregar os discos do Mongo ou Cassandra, o Redis gere estas atualizações em memória, permitindo que milhares de utilizadores interajam simultaneamente com latência sub-milisegundo.

Quando o utilizador final visualiza a aplicação, o círculo fecha-se:

1. O Cassandra entrega a lista de IDs das publicações (o Feed).
2. O Redis fornece instantaneamente o número de likes para esses IDs.
3. O MongoDB é consultado apenas quando o utilizador clica num post específico para ver os detalhes completos e comentários, otimizando o tráfego de rede e a carga nos servidores.


— escrita (write path) —
Utilizador (App)
Clica em "Publicar Post"
{ book title, author, content, rating, status }
Serviço da API (Backend)
Recebe e valida os dados
1.º gravar
MongoDB
Persistência de Conteúdo
db.posts.insertOne()
devolve → ObjectId (_id)
2.º escritas paralelas
Cassandra
Fan-out on Write
INSERT INTO timeline
post_id = timeuuid
Redis
Inicialização de Cache
HSET like:{post_id}
count 0
— leitura (read path) —
Ver detalhes
Carregar feed
Mostrar likes
Utilizador (App) — resultado
Feed · Detalhes do post · Contador de likes